🚀 Collections Framework Masterclass

Chapter 11 — Queue Interface (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`Queue` is one of the core Collection interfaces and is heavily used in **multithreading, messaging, task processing, BFS, scheduling, and producer-consumer systems**.

The most important thing to understand is:

> **Queue normally follows FIFO — First In, First Out — but not every Queue implementation guarantees FIFO ordering.**

That exception becomes important when we study `PriorityQueue`.

---

1. What is Queue?

`Queue` represents a collection designed primarily for holding elements before they are processed.

Package:

```
java.util.Queue
```

Hierarchy:

```
Iterable
   ↓
Collection
   ↓
Queue
   ├── LinkedList
   ├── PriorityQueue
   └── ArrayDeque
```

Some concurrent implementations include:

```
BlockingQueue
   ├── ArrayBlockingQueue
   ├── LinkedBlockingQueue
   └── PriorityBlockingQueue
```

We'll cover those later.

---

2. FIFO Principle

The typical Queue behavior is:

```
First In
   ↓
First Out
```

Example:

```
10 → 20 → 30 → 40
↑              ↑
head           tail
```

Remove:

```
10
```

Then:

```
20 → 30 → 40
```

This is similar to a real-world queue at a counter.

---

3. Queue Does NOT Always Mean FIFO

This is an important interview nuance.

A normal FIFO Queue processes elements in insertion order.

But implementations can use different ordering policies.

For example:

```
PriorityQueue
```

orders elements according to their priority/natural ordering rather than insertion order.

So say:

> "Queue is designed for processing elements in some ordering policy; FIFO is the standard Queue behavior, but implementations such as PriorityQueue can use priority ordering."

That's a stronger interview answer.

---

4. Core Queue Methods

Queue has three important method pairs:

```
Insertion
add()     / offer()

Removal
remove()  / poll()

Inspection
element() / peek()
```

This is extremely important for interviews.

---

5. add() vs offer()

Both attempt to insert an element.

```
queue.add(10);
```

versus:

```
queue.offer(10);
```

The difference appears when insertion cannot be performed.

Method	Failure behavior
`add()`	Throws exception
`offer()`	Returns `false`

For an unbounded Queue such as `LinkedList` or `ArrayDeque`, capacity failure normally isn't an issue.

The distinction becomes especially useful with **bounded queues**.

---

6. remove() vs poll()

Both remove the head.

```
queue.remove();
```

versus:

```
queue.poll();
```

If the Queue is empty:

```
remove() → NoSuchElementException

poll()   → null
```

Remember:

```
remove() → exception
poll()   → null
```

---

7. element() vs peek()

Both inspect the head without removing it.

```
queue.element();
```

versus:

```
queue.peek();
```

If empty:

```
element() → NoSuchElementException

peek()    → null
```

Remember:

```
element() → exception
peek()    → null
```

---

8. The Complete Method Table

Operation	Exception Version	Special-Value Version
Insert	`add(e)`	`offer(e)`
Remove	`remove()`	`poll()`
Inspect	`element()`	`peek()`

Easy memory trick:

```
add     → offer
remove  → poll
element → peek
```

The right-hand methods are generally safer when you want to handle failure without exceptions.

---

9. Basic Example

```
Queue<String> queue =
    new LinkedList<>();

queue.offer("A");
queue.offer("B");
queue.offer("C");

System.out.println(queue.poll());
System.out.println(queue.peek());
```

Output:

```
A
B
```

After `poll()`:

```
B → C
```

---

10. Queue Does Not Provide Index-Based Access

Unlike:

```
List
```

Queue doesn't provide:

```
get(index)
```

Why?

Because Queue focuses on:

```
Insert
Remove head
Inspect head
```

rather than positional access.

---

11. Queue Implementations

The important implementations for interviews are:

```
LinkedList
PriorityQueue
ArrayDeque
```

And later:

```
BlockingQueue implementations
```

---

12. LinkedList as Queue

Since LinkedList implements `Queue`:

```
Queue<Integer> queue =
    new LinkedList<>();
```

You can use:

```
offer()
poll()
peek()
```

Internally, LinkedList can efficiently add/remove at its ends.

---

13. ArrayDeque as Queue

`ArrayDeque` is usually a better general-purpose Queue than LinkedList.

```
Queue<Integer> queue =
    new ArrayDeque<>();
```

Advantages:

- Array-based
- Efficient
- No node allocation per element
- Good cache locality
- No synchronization overhead

For ordinary single-threaded FIFO queues, `ArrayDeque` is often the preferred choice.

---

14. ArrayDeque vs LinkedList

Feature	ArrayDeque	LinkedList
Internal structure	Resizable array/deque	Doubly linked list
Queue operations	Efficient	Efficient
Memory overhead	Lower	Higher
Cache locality	Better	Worse
Null elements	❌	✅
Typical choice	Preferred	Useful when List behavior is also needed

One important point:

> `ArrayDeque` does not permit `null`, because `null` is used to represent an empty result for methods such as `poll()` / `peek()`.

---

15. PriorityQueue

`PriorityQueue` is where many candidates make a mistake.

It is a Queue, but it is **not FIFO**.

```
Queue<Integer> queue =
    new PriorityQueue<>();

queue.offer(30);
queue.offer(10);
queue.offer(20);
```

The head is:

```
10
```

because the smallest element has the highest priority under natural ordering.

---

16. PriorityQueue Internal Structure

PriorityQueue uses a:

```
Binary Heap
```

By default:

```
Min-Heap
```

Conceptually:

```
        10
       /
     30    20
```

The smallest element is at the head.

We'll cover PriorityQueue in detail later.

---

17. Queue Ordering Comparison

Implementation	Ordering
LinkedList	FIFO
ArrayDeque	FIFO
PriorityQueue	Priority
BlockingQueue variants	Depends on implementation

---

18. Queue with Custom Objects

Suppose:

```
class Task {
    int priority;
    String name;
}
```

A priority-based Queue can use:

```
Queue<Task> tasks =
    new PriorityQueue<>(
        Comparator.comparingInt(
            t -> t.priority
        )
    );
```

Now tasks are processed according to the comparator.

This is a common real-world pattern.

---

19. Queue Use Cases

Queue is ideal when work must be processed progressively.

Task Processing

```
Request 1
Request 2
Request 3
       ↓
     Queue
       ↓
   Worker
```

Messaging

```
Producer
   ↓
 Queue
   ↓
Consumer
```

BFS

Breadth-First Search uses a Queue.

```
Start
  ↓
Queue
  ↓
Process level by level
```

Request Buffering

```
Incoming Requests
       ↓
      Queue
       ↓
    Workers
```

---

20. Producer-Consumer Pattern

A classic interview scenario:

```
Producer
   ↓
  Queue
   ↓
Consumer
```

Producer:

```
queue.offer(task);
```

Consumer:

```
Task task =
    queue.poll();
```

For multithreaded production systems, a regular `ArrayDeque` is not enough because it isn't thread-safe.

Use a concurrent queue such as a `BlockingQueue`.

---

21. BlockingQueue

`BlockingQueue` is part of:

```
java.util.concurrent
```

It extends Queue and adds blocking behavior.

Example:

```
BlockingQueue<Task> queue =
    new ArrayBlockingQueue<>(100);
```

Producer:

```
queue.put(task);
```

Consumer:

```
Task task =
    queue.take();
```

If the queue is full:

```
put()
↓
wait
```

If the queue is empty:

```
take()
↓
wait
```

This makes BlockingQueue particularly useful for producer-consumer architectures.

---

22. Queue Method Families in Concurrent Queues

With bounded queues, the method choice becomes especially meaningful:

```
add()      → exception on failure
offer()    → false on failure
put()      → waits
offer(e,t) → waits up to timeout
```

Removal:

```
remove()   → exception
poll()     → null
take()     → waits
poll(t)    → waits up to timeout
```

This distinction is very useful in concurrency interviews.

23. Queue vs Stack

Interview Favorite ⭐⭐⭐⭐⭐

Queue:

```
FIFO
```

Stack:

```
LIFO
```

Example:

```
Queue

10 → 20 → 30

remove → 10
```

Stack:

```
10
20
30 ← top

pop → 30
```

Modern Java generally uses:

```
Deque
```

for stack behavior rather than the legacy `Stack` class.

---

24. Queue vs Deque

`Queue` normally focuses on:

```
Insert at tail
Remove from head
```

`Deque` supports both ends:

```
Front ⇄ Rear
```

Therefore Deque can behave as:

```
Queue
+
Stack
```

Example:

```
Deque<Integer> deque =
    new ArrayDeque<>();
```

Queue:

```
deque.offerLast(10);
deque.pollFirst();
```

Stack:

```
deque.push(10);
deque.pop();
```

---

25. Queue vs List

Feature	Queue	List
Primary purpose	Processing order	Ordered collection
Index access	❌	✅
`get(index)`	❌	✅
Head operations	Core functionality	Not the focus

Feature	Queue	List
FIFO semantics	Common	No

---

26. Null Elements

This depends on the implementation.

LinkedList

Allows `null`.

```
Queue<String> q =
    new LinkedList<>();

q.offer(null);
```

ArrayDeque

Does not allow `null`.

PriorityQueue

Does not allow `null`.

Important reason:

For methods such as `poll()` and `peek()`, `null` is used to indicate that the queue is empty.

---

27. Queue is an Interface

Prefer programming to the interface:

```
Queue<Task> queue =
    new ArrayDeque<>();
```

rather than:

```
ArrayDeque<Task> queue =
    new ArrayDeque<>();
```

unless you specifically need ArrayDeque-only operations.

This makes the implementation easier to replace.

---

28. Time Complexity

For common implementations:

Operation	ArrayDeque	LinkedList	PriorityQueue
Offer	O(1) amortized	O(1)	O(log n)
Poll	O(1)	O(1)	O(log n)
Peek	O(1)	O(1)	O(1)
Search/contains	O(n)	O(n)	O(n)

PriorityQueue has O(log n) insertion/removal because the heap may need reordering.

---

29. Common Mistakes

Mistake 1

> Queue always means FIFO.

Not universally.

`PriorityQueue` uses priority ordering.

Mistake 2

Using `remove()` without considering an empty Queue.

Safer alternative:

```
queue.poll();
```

when `null` is an acceptable empty indication.

Mistake 3

Using LinkedList as the default Queue.

For a normal single-threaded Queue:

```
ArrayDeque
```

is generally a better choice.

Mistake 4

Using ArrayDeque for a multi-threaded producer-consumer system.

It is not thread-safe.

Use an appropriate concurrent Queue, commonly a `BlockingQueue`.

Mistake 5

Using `PriorityQueue` when you need insertion order.

PriorityQueue does not preserve FIFO ordering.

---

30. ⭐ Interview Scenario

Requirement

You need a normal FIFO Queue for processing tasks in a single-threaded application.

Best choice?

```
Queue<Task> queue =
    new ArrayDeque<>();
```

Why?

```
FIFO
+
Efficient
+
Low memory overhead
+
No synchronization needed
```

---

31. ⭐ Interview Scenario

Requirement

Multiple producer threads submit tasks and multiple consumer threads process them.

The Queue should wait when:

- Full → producer waits
- Empty → consumer waits

Use:

```
BlockingQueue<Task>
```

For example:

```
BlockingQueue<Task> queue =
    new ArrayBlockingQueue<>(100);
```

---

32. ⭐ Interview Scenario

Requirement

You need to repeatedly process the smallest value.

Use:

```
PriorityQueue<Integer>
```

because it provides priority ordering.

---

33. Frequently Asked Interview Questions

Q1. What is Queue?

A Collection designed primarily for holding elements before processing, usually with FIFO semantics.

Q2. What is FIFO?

First In, First Out.

Q3. Difference between `add()` and `offer()`?

```
add()
→ exception if insertion fails

offer()
→ false if insertion fails
```

Q4. Difference between `remove()` and `poll()`?

```
remove()
→ exception if empty

poll()
→ null if empty
```

Q5. Difference between `element()` and `peek()`?

```
element()
→ exception if empty
```

```
peek()
→ null if empty
```

Q6. Is PriorityQueue FIFO?

No.

It orders elements according to priority.

Q7. Which Queue implementation is generally preferred for ordinary FIFO behavior?

`ArrayDeque`.

Q8. Is LinkedList a Queue?

Yes. It implements `Deque`, which extends `Queue`.

Q9. Is ArrayDeque thread-safe?

No.

Q10. Which Queue should you use for producer-consumer communication?

Usually an appropriate `BlockingQueue`, depending on the requirements.

Q11. Can Queue contain null?

Depends on implementation.

`LinkedList` allows it; `ArrayDeque` and `PriorityQueue` do not.

Q12. What is the difference between Queue and Deque?

Queue primarily models one-ended insertion/removal behavior, while Deque supports insertion and removal at both ends.

---

34. ⭐ Quick Interview Test

Given:

```
Queue<Integer> q =
    new ArrayDeque<>();

q.offer(10);
q.offer(20);
q.offer(30);

System.out.println(q.peek());
System.out.println(q.poll());
System.out.println(q.peek());
```

Output:

```
10
10
20
```

Because:

```
offer → add at tail
peek  → inspect head
poll  → remove head
```

---

35. Decision Framework

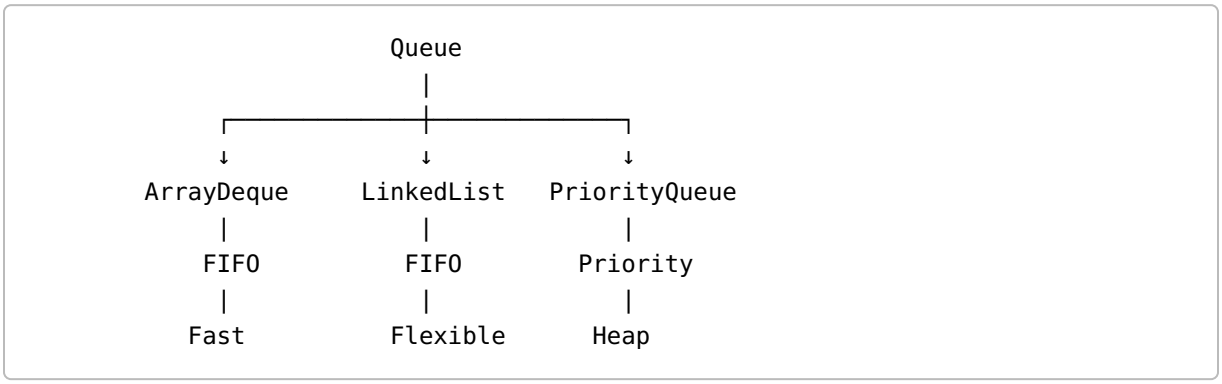
```
Need a Queue?
      ↓
Normal FIFO?
   ↙       ↘
 YES        NO
 ↓           ↓
Need       Priority?
concurrency   ↓
 ↓         PriorityQueue
 NO
```

```
 ↓
ArrayDeque
```

For concurrency:

```
Need producer/consumer blocking?
             ↓
           YES
             ↓
       BlockingQueue
```

🧠 Final Mental Model



Method pairs:

```
INSERT
add()      → exception
offer()    → special value

REMOVE
remove()   → exception
poll()     → special value

INSPECT
element()  → exception
peek()     → special value
```

For modern Java:

```
Normal Queue
     ↓
ArrayDeque

Concurrent producer/consumer
     ↓
BlockingQueue

Priority processing
     ↓
PriorityQueue
```

🔥 Chapter Summary

Concept	Key Point
Queue	Holds elements for processing
Typical behavior	FIFO
`add()`	Insert, exception on failure
`offer()`	Insert, special-value failure
`remove()`	Remove head, exception if empty
`poll()`	Remove head, `null` if empty
`element()`	Inspect head, exception if empty
`peek()`	Inspect head, `null` if empty
ArrayDeque	Preferred general-purpose FIFO Queue
LinkedList	Queue + Deque + List
PriorityQueue	Priority ordering
BlockingQueue	Concurrent producer-consumer use
Thread-safe?	Depends on implementation

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] What is Queue?
- [ ] What does FIFO mean?
- [ ] Why doesn't PriorityQueue follow FIFO?
- [ ] `add()` vs `offer()`?
- [ ] `remove()` vs `poll()`?
- [ ] `element()` vs `peek()`?
- [ ] ArrayDeque vs LinkedList?
- [ ] Queue vs Deque?
- [ ] Queue vs Stack?
- [ ] Why is ArrayDeque preferred over LinkedList for ordinary queues?
- [ ] Why can't ArrayDeque contain null?
- [ ] What is BlockingQueue?
- [ ] Which collection would you use for producer-consumer architecture?
- [ ] Which collection would you use for priority-based processing?

🚀 Next Chapter

Chapter 12 — Deque Interface (Complete Deep Dive)

We'll focus on what's new:

- Queue + Stack in one interface
- `addFirst()` / `addLast()`
- `offerFirst()` / `offerLast()`
- `pollFirst()` / `pollLast()`
- `peekFirst()` / `peekLast()`
- `ArrayDeque` internals
- Circular-array concept
- Stack implementation using Deque
- Queue implementation using Deque
- ArrayDeque vs LinkedList
- Why `Stack` is legacy
- Real-world use cases
- 20+ interview questions